

해커톤 AI 협업

처음 만나도 팀이 된다

해커톤을 위한 AI 협업 시스템

전시진 (SIREAL) · 2026. 06. 23

오늘의 흐름

01 소통 창구 만들기

02 아이디어션

03 아이디어 선별

04 문서화

05 구현

06 오류수정

07 발표준비

FAQ 자주 묻는 질문

강의 목적

이 강의는 무엇을 위한 것인가

- 개발자 + 비개발자가 섞인 팀이 함께 움직이는 방법
- 코딩을 잘 못해도 **AI와 함께라면** 해커톤 프로젝트를 끝까지 완성할 수 있다
- 핵심은 직접 만드는 것이 아니라 **AI에게 잘 시키는 것**
- 잘 시키려면 **명확한 기록과 문서**가 반드시 필요하다

핵심 원칙

기억할 핵심 두 가지

① AI에게 "잘 시키는" 시대

- 직접 만드는 게 아니라 AI에게 시키는 시대
- 잘 시키려면 **기록과 문서**가 필요하다
- 결과물의 품질 = **지시문의 품질**

② 모든 것은 노선에 기록으로

- 모든 대화·자료·결정은 **노선에 기록**
- 그 기록이 AI가 일하는 **재료(컨텍스트)**
- 기록하지 않은 것 = 존재하지 않는 것

전체 구조

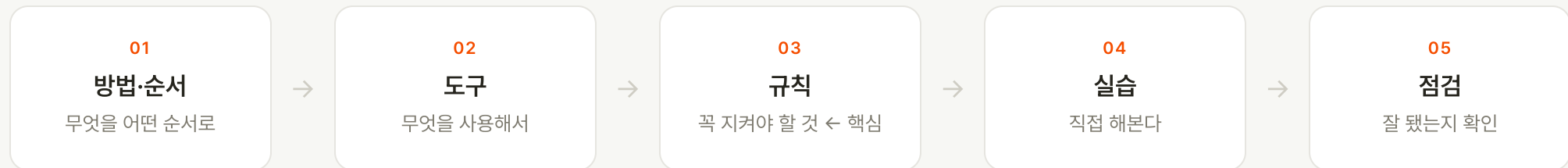
전체 흐름



↔ 반복 루프 — 좋은 아이디어가 나올 때마다 바로 덤리서치를 돌린다

- 아이디어션 ↔ 아이디어 선별은 순서가 아니라 반복 루프다
- 나머지 단계는 순서대로 진행한다

각 단계마다 같은 5요소



이것만 지켜도 절반은 성공

소통 창구 만들기

모든 기록이 모이는 팀의 단 하나의 창구

01

소통 창구 만들기

팀 역할 분담

역할	핵심 책임	3명	4명	5명
팀장	전체 일정 조율, 회의 진행, 의사결정, 발표 주도			
기획자	아이디어 발전, 요구사항 정리(PRD), 답리서치 주도			
개발자	구현, 오류수정, GitHub · Vercel 관리			
디자이너	UI/UX, 슬라이드 제작, 시각자료			
리서처	시장·사용자 조사, 발표 스크립트 작성			

3명 — ① 팀장+기획자 ② 개발자 ③ 디자이너+리서처

4명 — ① 팀장 ② 기획자+리서처 ③ 개발자 ④ 디자이너

5명 — 각 역할 전담 1인

겸임 전담

소통 창구 만들기

방법 · 순서

- 1 노션 **페이지** 생성 — 팀 전용 공간을 만든다
- 2 **팀원 전원 초대** — 한 명이라도 빠지면 그 사람의 기록이 단절된다
- 3 페이지 안에 **데이터베이스** 생성 — 문서를 종류별로 쌓을 그릇
- 4 각종 문서를 **종류별**로 정리 — 회의록 / 자료 / 할 일 / 문서

소통 창구 만들기

도구

- **Notion** — 팀 문서의 단일 허브
- 복잡한 DB 구조는 필요 없다 — 단순할수록 팀이 잘 쓴다
- `종류` 속성(**select**) 하나 만 달면 충분 — 회의록 / 자료 / 할 일 / 문서
- 나중에 **MCP 연결** 시 "회의록만 모아줘" 같은 AI 활용이 가능

소통 창구 만들기

규칙

- 1 매 회의 시작 시 AI 회의 노트를 켜다 ← 가장 중요
- 2 모든 자료·기록은 노션 **한곳**에 — 카톡·슬랙으로 분산 금지
- 3 제목은 나중에 봐도 알아볼 수 있게 — "회의1", "메모" 금지
- 4 `종류` 속성 **하나만** — 복잡한 태그는 쓸 일 없다
- 5 좋은 의견·결정은 반드시 **문서로** 남긴다

⚠ AI 노트를 켜지 않은 회의 = 흔적이 사라진다

소통 창구 만들기

실습 · 점검

실습

- 노션 페이지 만들고 팀원 **전원** 초대
- 데이터베이스 생성 → `종류` 속성 추가
- 첫 항목(오늘 회의록) 올려보기

점검

- 팀원이 전원 초대됐는가?
- 데이터베이스에 `종류` 속성이 있는가?
- 첫 문서가 노션 **한곳**에 저장됐는가?

아이데이션

판단은 보류, 양으로 승부 — 좋은 아이디어엔 즉시 답리서치

02

방법 · 순서

- 1 문제 정의 — 우리 팀이 풀 문제가 무엇인지 먼저 좁힌다
- 2 자료 수집 — 관련 뉴스·데이터·사례를 모은다
- 3 회의로 아이디어 쏟아내기 — 비판 없이 양으로 승부
- 4 괜찮은 아이디어 → 즉시 딥리서치 시작 ← 핵심

딥리서치는 시간이 걸린다. 아이디어가 나올 때마다 바로 돌려두면 아이데이션을 계속하면서 결과를 기다릴 수 있다.

아이데이션

도구

회의 기록

- 노션 AI 노트 (우선) — 자동 기록·요약
- 클로바노트 (대안) — 웹 접속 가능

딥리서치 (즉시)

- Gemini Deep Research — 긴 리포트, 출처 포함
- ChatGPT Deep Research — 단계별, 근거 풍부
- Claude — 문서 분석·구조화 평가
- Perplexity — 빠른 팩트체크

아이데이션

규칙

- 회의 시작과 동시에 **AI 노트북터** 켜다
- 다른 사람 아이디어 **비판 금지** — 지금은 아이디어를 늘리는 단계
- **긍정·확장만** — "그 아이디어에서 이렇게 가면 어떨까?"
- 판단 보류, **양으로 승부** — 아이디어 10개 중 1개면 충분
- 떠오르면 **즉시 기록** — 기록하지 않으면 사라진다
- **괜찮다 싶으면 즉시 딥리서치** — 기다리지 않는다

아이데이션

실습 · 점검

실습

- 클로바노트(또는 노션 AI 노트)로 아이디어 회의 진행
- 회의 내용을 노션 문서로 정리
- 괜찮은 아이디어 1~2개 → 딥리서치 시작

점검

- 회의 내용이 노션에 정리됐는가?
- 아이디어별 코멘트가 달렸는가?
- 딥리서치가 최소 1개 이상 돌아가고 있는가?

아이디어 선별

답리서치 결과를 심사 기준에 대입해 1개를 확정한다

03

아이디어 선별

방법 · 순서

아이데이션 단계에서 딥리서치를 반복했다면, 이미 근거 자료가 쌓여 있다.

- 1 딥리서치 결과를 **후보별로 정리**
- 2 **심사 기준 5가지** 적용 → 항목별 점수
- 3 **우선순위** 결정 — 점수 + 팀 반응 ("이거 좋은데요?")
- 4 **1개 확정** → 선정 이유를 노선에 기록

아이디어 선별

심사 기준 5가지

- 1 문제 정의 및 사회적 필요성 — 이 문제가 실제로 존재하고 중요한가?
- 2 창의성 및 독창성 — 기존 솔루션과 어떻게 다른가?
- 3 AI 활용의 적절성 및 기술 타당성 — AI를 의미있게 쓰고 있는가?
- 4 실현 가능성 및 완성도 — 24~48시간 내에 MVP를 만들 수 있는가?
- 5 기대 효과 및 파급력 — 실제로 적용됐을 때 어떤 변화가 생기는가?

아이디어 선별

딥리서치 도구

도구	특징	언제 쓰면 좋은가
Gemini Deep Research	긴 리포트, 출처 링크 자동 포함	시장·경쟁사 분석
ChatGPT Deep Research	단계별 리서치, 근거 풍부	문제·사회적 필요성 검증
Claude	긴 문서 분석·구조화 평가	심사 기준별 평가 요청
Perplexity	빠른 팩트체크, 최신 정보	아이디어 첫 검증

예시 프롬프트 → Example_prompt/ 폴더 참고

아이디어 선별

규칙

- 심사위원 관점으로 본다 — "내가 보기엔 좋은데"가 아니라 기준으로
- "이거 좋은데요?" 반응을 신호로 포착 — 이유를 반드시 기록
- 딥리서치는 아이디어 나올 때 즉시 — 선별 단계까지 기다리지 않는다
- 결과를 기다리는 동안 다음 아이디어를 발전시킨다
- 선택한 이유를 반드시 문서에 남긴다

아이디어 선별

실습 · 점검

실습

- 아이디어 1개 → Example_prompt/ 에서 프롬프트 선택
- 답리서치 도구에 입력 → 결과를 심사 기준표에 대입
- 후보 2~3개 점수화 → 팀 합의로 1개 선택
- 선정 이유를 노선에 한 단락으로 기록

점검

- 심사 기준표가 채워졌는가?
- 선정 이유가 문서로 남았는가?
- 기준이 "느낌"이 아니라 근거로 설명되는가?

문서화

AI가 읽기 좋은 마크다운 문서 — 구현과 발표의 재료

04

문서화

방법 · 순서

- 1 문서 목록 잡기 — 무엇을 문서로 만들지 먼저 리스트업
- 2 마크다운으로 작성 — AI가 가장 잘 읽는 형식
- 3 팀 검토 — AI가 읽기 좋은 구조인가? 빠진 내용은?
- 4 docs/ 폴더에 묶기 — 하나의 폴더로 관리
- 5 지속 업데이트 — 구현 중에도 변경사항 즉시 반영

문서화

어떤 문서를 만들까

우선순위	문서	역할
필수	PRD	무엇을 — 범위·기능·기대효과
필수	README	프로젝트 한 줄 소개, 실행 방법
권장	화면·기능 정의	구현 단위 (한 화면 = 한 문서)
권장	Changelog	무엇을 만들고 고쳤는지 이력
선택	디자인 가이드	색·폰트·컴포넌트 톤
선택	API 명세	백엔드-프론트 연동 기준

모든 문서 → 프로젝트 루트의 **docs/** 폴더 하나에 모은다

문서화

도구 · 규칙

도구 — 마크다운 에디터, CLAUDE, CURSOR

- **마크다운**으로 작성 — AI가 가장 잘 읽는 형식이다
- AI에 너무 의존하면 **문장만 늘어난다** — 핵심만 담은 짧은 문서
- **2,000줄 이내**로 유지 — 짧을수록 AI가 더 잘 활용한다
- 길어지면 **주제별로 쪼개기** — 하나의 문서가 너무 많으면 AI도 헷갈린다

문서화

기억할 것

- AI는 **결정을 대신하지 않는다**
- 프롬프트를 입력하는 사람의 기반에 맞춰 결과를 낼 뿐이다
- 방향·결정은 팀이 먼저 한다. AI는 그 결정을 **실행하는 도구**다
- 의사결정 시 → 팀원 의견 + 딥리서치 근거 + 심사 기준으로 결정한다

실습 — PRD를 마크다운 1장으로 작성 (기능 3~5개, 기대효과 포함)

점검 — "AI에게 이 문서를 주면 바로 구현할 수 있는가?" 상호 리뷰

구현

docs/를 붙이고 Agent mode로 — 잘 시키는 게 전부다

05

구현

방법 · 순서

- 1 **Repo 생성** (GitHub) + **배포 환경 준비** (Vercel 등)
- 2 **docs/ 폴더 확인** — PRD, README 최소 준비
- 3 **구현 범위 확정** — PRD 기준으로 이번 세션에 만들 기능 1개 선택
- 4 **Agent mode**로 구현 — docs/ 파일을 @첨부하고 지시
- 5 **브라우저에서 직접 확인** — AI 말만 믿지 않는다
- 6 **Commit + Push** — 확인된 것만 커밋

구현

Agent mode가 기본

Agent mode (기본)

- docs/ @첨부 → "OO 기능 구현해줘"
- 방향이 명확하면 바로 구현 시작

Plan mode (이럴 때 사용)

- 여러 페이지·기능을 **한꺼번에** 구현할 때
- 동시에 **여러 단계**를 거쳐야 할 때
- **복잡한 기능**이라 범위 파악이 먼저 필요할 때
- → 계획 확인 후 Agent 전환

Commit + Push는 **팀원 누구나** 한다 — 개발자 전담이 아니다 | 기능 추가 시 **New 세션**

구현

구현할 때 docs/를 이렇게 쓴다

문서	AI에게 넘길 때 역할
PRD	무엇을 왜 만드는지 — 범위·기능
화면·기능 정의	이번 세션에서 만들 것 — 구현 단위
README	프로젝트 맥락 — 스택·구조
디자인 가이드	UI 톤·컴포넌트 스타일
Changelog	지금까지 만든 것 — 충돌 방지

구현

AI에게 잘 시키는 법 ①

이제는 직접 만드는 게 아니라 AI에게 시키는 시대다.

- 1 구체적으로 — "로그인 버튼 만들어줘"가 아니라 동작·결과까지 명확히
- 2 맥락을 함께 — 관련 문서·파일을 @첨부하고 무엇을 만들지 알려준다
- 3 한 번에 하나씩 — 큰 작업은 기능·화면 단위로 쪼개서 지시한다

구현

AI에게 잘 시키는 법 ②

- 4 **제약을 명시** — 언어·라이브러리·스타일·형식 조건을 분명히
- 5 **기대 결과를 보여준다** — 예시 화면, 원하는 출력 형태 제시
- 6 **안 되면 다시** — 결과 확인 후 부족하면 지시문을 고쳐서 반복

결과물의 품질은
지시문의 품질이 결정한다.

오류수정

오류는 반드시 생긴다 — 재현하고, 기록하고, AI에게 시킨다

06

오류수정

방법 · 순서

- 1 문제 발견·기록 — 어떤 화면, 어떤 순서, 어떤 결과가 나왔는지
- 2 2~3회 재현 — 같은 조건에서 반복되는지 확인
- 3 docs/와 대조 — 버그인지, 문서 오류인지, 기획 변경인지 구분
- 4 Agent mode로 수정 지시 — 증상·재현 순서·기대 결과를 함께
- 5 브라우저 확인 → Changelog 업데이트 → Commit + Push

오류수정

개발자가 있다면, 없다면

가능하면 **개발자가 코드를 직접 보면서** 수정하는 것이 가장 좋다.
팀에 개발자가 없다면, 비개발자가 AI와 함께 주도한다.

상황	누가 어떻게
개발자가 있을 때	개발자가 코드·에러 로그를 직접 보고 수정 주도
개발자가 없을 때	비개발자가 발견 → 재현 → AI 지시 → 확인 → Commit + Push

오류수정

지시문 작성 요령 ①

- 1 **증상을 구체적으로** — "안 돼요"가 아니라 "로그인 버튼 클릭 시 흰 화면이 뜨고 아무 반응이 없다"
- 2 **기대 결과는 docs/에서** — "PRD 3번 기능 기준으로 로그인 성공 시 대시보드로 이동"
- 3 **재현 순서 그대로** — 1단계: 이메일 입력, 2단계: 비밀번호 입력, 3단계: 로그인 클릭
- 4 **한 번에 하나만** — 오류 2개를 한 번에 지시하면 AI가 하나를 빠뜨린다

오류수정

지시문 작성 요령 ②

- 5 스크린샷·화면 녹화 첨부 — 말보다 직접 보여주는 게 빠르다
- 6 직접 확인 — AI가 "수정했습니다"라고 해도 브라우저에서 반드시 확인
- 7 확인 후 **Commit + Push** — 개발자에게만 말기지 않는다
- 8 고치기 ≠ 기능 추가 — 새 기능은 New 세션에서 5번 구현 흐름으로

실습 — 오류 1개 재현 → 지시문 → Agent 수정 → 확인 → Changelog → Push

발표준비

docs/가 이미 재료다 — 목차 → 내용 → 디자인 순서로

07

발표준비

3단계 순서

STEP 1

문서 정리 · 목차

PRD·Changelog·README에서 핵심 메시지 추출,
목차 먼저 확정



STEP 2

세부 내용 작성

목차별 문장·수치·시연 포인트, AI에게 docs/ 첨부 후
요청



STEP 3

디자인 적용

내용 완성 후 디자인 가이드에 맞춰 슬라이드 완성

목차·내용 **먼저**, 디자인은 **마지막** — 순서를 바꾸면 내용이 디자인에 끌려다닌다

발표준비

docs/ → 발표 재료로 활용

문서	발표에서 쓰이는 내용
PRD	문제 정의 · 핵심 기능 · 기대 효과
Changelog	"우리가 실제로 만든 것" — 구현 증거
README	프로젝트 한 줄 소개 · 기술 스택
딥리서치 결과	시장 규모 · 사회적 필요성 근거 수치

도구 — Gemini, NotebookLM, Claude, Canva, Gamma 등

발표준비

규칙 · 실습

- docs/에서 **시작** — 새로 쓰지 않는다
- **목차 먼저** 확정하고 내용을 채운다
- 슬라이드 생성은 쉽다 — **안에 들어가는 내용이 항상 어렵다**
- AI 결과 ↔ docs/ 대조 — 사실과 다른 내용은 직접 수정
- 발표 전 **시연 리허설 + 시간 맞추기** 필수

실습 — PRD·Changelog @첨부 → 목차 5~7장 확정 → 내용 채우기 → 슬라이드 1장 디자인

자주 묻는 질문

현장에서 나오는 질문, 미리 답한다

FAQ

Q. 소통 창구 만들 시간이 어딴어요?

바쁠수록 더 필요하다.

- 소통 창구가 없으면 "아까 누가 뭐 말했더라"를 반복하다 시간을 다 쓴다
- 노션 페이지 + 팀원 초대 = 5분
- 그 5분이 이후 문서화 → AI 구현 → 발표준비까지 전부 연결되는 기반이 된다
- 바빠서 안 만들면, 더 바빠진다

Q. 비개발자는 뭘 하나요?

할 일이 더 많아진다.

- 문제 정의 · 딥리서치 · PRD 문서화
- 지시문(프롬프트) 작성 — 결과물의 품질을 결정하는 핵심
- 오류 수정 주도 — 재현 · Agent 지시 · 확인
- Commit + Push — 확인된 코드는 누구나 올린다
- 발표 준비 — 스토리라인 · 슬라이드 · 시연

Q. AI 결과를 그대로 믿어도 되나요?

안 된다.

- AI는 **결정을 대신하지 않는다** — 입력한 사람의 기반에 맞춰 결과를 낼 뿐
- 코드 → **브라우저에서 직접 실행·테스트**
- 문서·리서치 결과 → **출처·사실 확인**
- 딥리서치 통계 → **다른 도구로 교차 확인**

"AI가 했으니 맞겠지"는 발표장에서 가장 위험한 생각이다

Q. AI를 한 번도 안 써봤어요

AI 노트 **켜기**부터 시작한다.

- 1 회의할 때 노션 AI 노트 또는 클로바노트를 켜다
- 2 말하면 기록이 남는다 — **가장 쉬운 AI 활용**이다
- 3 익숙해지면 → 검색 → 답리서치 → 지시문 작성 순으로 확장
- 4 팀에서 **먼저 시도하는 사람**이 생기면 나머지도 따라간다

Q. 비개발자도 Commit + Push를?

한다. 해커톤 팀원 누구나.

- 오류 재현 → docs/ 기준 Agent 지시 → 브라우저 확인
- 확인됐으면 → Changelog 업데이트 → **Commit + Push**
- 개발자만 올리는 것이 아니다 — **확인한 사람이 올린다**

Commit + Push는 코딩이 아니다. "**내가 확인한 변경사항을 저장한다**"는 행위다.

정리

소통 창구 → 아이디어션 ↔ 선별(답리서치) → 문서화 → 구현 → 오류수정 → 발표



기록

모든 대화·결정은 노선에 남긴다



문서화

마크다운 docs/가 구현과 발표의 재료다



잘 시키기

결과물의 품질은 지시문의 품질이 결정한다

질문 · 실습 시간